

Implementing WordPiece

এখানে দেখানো হচ্ছে, কীভাবে **WordPiece algorithm** কোড দিয়ে বানানো যায়।

তবে একটা কথা শুরুতেই বলা হয়েছে:

এটা শেখার জন্য বানানো **implementation**।

মানে, **concept** বোঝানোর জন্য।

অনেক বড় **dataset** বা **real-world** বড় **corpus**-এর জন্য এটা **practical** না।

কোন **corpus** ব্যবহার করা হয়েছে?

এখানে আগের BPE example-এর একই **corpus** নেওয়া হয়েছে:

```
corpus = [  
    "This is the Building-LLMs-from-scratch-In-Bangla Course.",  
    "This chapter is about tokenization.",  
    "This section shows several tokenizer algorithms.",  
    "Hopefully, you will be able to understand how they are trained and generate tokens.",  
]
```

এগুলো কয়েকটা **sentence**, যেগুলো থেকে WordPiece tokenizer train করা হবে।

Step 1: আগে **text**-কে **word**-এ ভাগ করা হচ্ছে

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
```

এখানে **bert-base-cased** tokenizer নেওয়া হয়েছে।

cased মানে:

- বড় হাতের অক্ষর আর ছোট হাতের অক্ষর আলাদা ধরা হবে
- যেমন **H** আর **h** এক জিনিস না

এই tokenizer ব্যবহার করা হচ্ছে **pre-tokenization** এর জন্য।

মানে, পুরো **sentence**-কে আগে ছোট ছোট **word/punctuation**-এ ভাঙা হবে।

Step 2: প্রতিটি word কতবার এসেছে সেটা count করা হচ্ছে

```
from collections import defaultdict
```

```
word_freqs = defaultdict(int)
for text in corpus:
    words_with_offsets = tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str(text)
    new_words = [word for word, offset in words_with_offsets]
    for word in new_words:
        word_freqs[word] += 1
```

এখানে কী হচ্ছে?

প্রতিটা sentence থেকে word বের করা হচ্ছে, তারপর কোন word কয়বার এসেছে সেটা count করা হচ্ছে।

যেমন output:

```
defaultdict(
  int, {'This': 3, 'is': 2, 'the': 1 .....}
)
```

এর মানে:

- **This** এসেছে 3 বার
- **is** এসেছে 2 বার
- **the** এসেছে 1 বার

অর্থাৎ, এখন tokenizer জানে কোন word কতবার আছে।

Step 3: alphabet তৈরি করা হচ্ছে

WordPiece-এর শুরুতে vocabulary খুব ছোট থাকে।
সেখানে থাকে:

- word-এর প্রথম letter
- আর word-এর পরের letterগুলো ## prefix সহ

এই code সেটা বানাচ্ছে:

```
alphabet = []
for word in word_freqs.keys():
    if word[0] not in alphabet:
        alphabet.append(word[0])
```

```
for letter in word[1:]:
    if f"#{letter}" not in alphabet:
        alphabet.append(f"#{letter}")
```

উদাহরণ:

যদি word হয় **Face**

তাহলে split হবে এমন:

- **F**
- **##a**
- **##c**
- **##e**

অর্থাৎ:

- প্রথম letter plain থাকবে
- বাকিগুলো **##** দিয়ে থাকবে

তারপর alphabet sort করা হয়েছে:

```
alphabet.sort()
```

Step 4: special tokens যোগ করা হচ্ছে

BERT-এর কিছু fixed token থাকে:

```
vocab = ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"] + alphabet.copy()
```

এগুলোর কাজ সংক্ষেপে:

- **[PAD]** → length equal করতে padding
- **[UNK]** → unknown word
- **[CLS]** → classification task-এ special token
- **[SEP]** → sentence separator
- **[MASK]** → masked language modeling-এর জন্য

এগুলো vocabulary-র শুরুতে যোগ করা হয়।

Step 5: প্রতিটি word character level-এ split করা হচ্ছে

```
splits = {
    word: [c if i == 0 else f"#{c}" for i, c in enumerate(word)]
    for word in word_freqs.keys()
}
```

এখানে প্রতিটি word-কে character-এ ভাঙা হচ্ছে।

যেমন:

`about` → `['a', '##b', '##o', '##u', '##t']`

মানে:

- প্রথম letter `a`
- তারপর `##b, ##o, ##u, ##t`

এটাই WordPiece-এর starting point।

Step 6: pair score বের করা হচ্ছে

এখন algorithm ঠিক করবে, কোন দুইটা token merge করলে ভালো হয়।

তার জন্য function:

```
def compute_pair_scores(splits):
    letter_freqs = defaultdict(int)
    pair_freqs = defaultdict(int)
    for word, freq in word_freqs.items():
        split = splits[word]
        if len(split) == 1:
            letter_freqs[split[0]] += freq
            continue
        for i in range(len(split) - 1):
            pair = (split[i], split[i + 1])
            letter_freqs[split[i]] += freq
            pair_freqs[pair] += freq
            letter_freqs[split[-1]] += freq

    scores = {
        pair: freq / (letter_freqs[pair[0]] * letter_freqs[pair[1]])
        for pair, freq in pair_freqs.items()
    }
    return scores
```

সহজভাবে এই function কী করে?

এটা দুইটা জিনিস count করে:

1. প্রতিটি token কতবার এসেছে
2. প্রতিটি pair কতবার এসেছে

তারপর score বের করে:

$$score = \frac{freq_of_pair}{freq_of_first \times freq_of_second}$$

অর্থাৎ, কোন pair merge করলে বেশি meaningful হবে সেটা বের করা হচ্ছে।

Step 7: score দেখে best pair বের করা হচ্ছে

```
pair_scores = compute_pair_scores(splits)
for i, key in enumerate(pair_scores.keys()):
    print(f"{key}: {pair_scores[key]}")
    if i >= 5:
        break
```

এখানে কিছু pair score print করা হয়েছে।

উদাহরণ:

```
('T', '##h'): 0.125
('##h', '##i'): 0.03409090909090909
('##i', '##s'): 0.02727272727272727
```

অর্থাৎ:

- T + ##h
- ##h + ##i
- ##i + ##s

এই pairগুলোর score বের হয়েছে।

এরপর best pair খোঁজা হচ্ছে:

```
best_pair = ""
max_score = None
```

```
for pair, score in pair_scores.items():
    if max_score is None or max_score < score:
        best_pair = pair
        max_score = score
```

এখানে সবচেয়ে বড় score-ওয়ালা pair নেওয়া হবে।

Output:

('a', '##b') 0.2

মানে প্রথম merge হবে:

a + ##b → ab

Step 8: নতুন merged token vocabulary-তে যোগ করা হচ্ছে

```
vocab.append("ab")
```

এখন vocabulary-তে **ab** যোগ হয়ে গেল।

Step 9: split-এর ভিতরে merge apply করা হচ্ছে

এখন শুধু vocab-এ token যোগ করলেই হবে না, existing split-গুলোর মধ্যেও merge লাগতে হবে।

তার জন্য function:

```
def merge_pair(a, b, splits):
    for word in word_freqs:
        split = splits[word]
        if len(split) == 1:
            continue
        i = 0
        while i < len(split) - 1:
            if split[i] == a and split[i + 1] == b:
                merge = a + b[2:] if b.startswith("##") else a + b
                split = split[:i] + [merge] + split[i + 2 :]
            else:
                i += 1
        splits[word] = split
    return splits
```

সহজভাবে এর কাজ:

যেখানে যেখানে **a** আর **##b** পাশাপাশি আছে, সেখানে merge করে **ab** বানায়।

যেমন:

```
splits = merge_pair("a", "##b", splits)
splits["about"]
```

Output:

```
['ab', '##o', '##u', '##t']
```

আগে ছিল:

```
['a', '##b', '##o', '##u', '##t']
```

এখন হয়েছে:

```
['ab', '##o', '##u', '##t']
```

Step 10: loop চালিয়ে vocabulary বড় করা হচ্ছে

```
vocab_size = 70
while len(vocab) < vocab_size:
    scores = compute_pair_scores(splits)
    best_pair, max_score = "", None
    for pair, score in scores.items():
        if max_score is None or max_score < score:
            best_pair = pair
            max_score = score
    splits = merge_pair(*best_pair, splits)
    new_token = (
        best_pair[0] + best_pair[1][2:]
        if best_pair[1].startswith("##")
        else best_pair[0] + best_pair[1]
    )
    vocab.append(new_token)
```

এখানে loop চলবে যতক্ষণ vocabulary size 70 না হয়।

প্রতিবার loop-এ যা হয়:

1. সব pair-এর score বের হয়
2. best pair বাছা হয়
3. সেটা merge করা হয়
4. নতুন token vocab-এ যোগ হয়

এভাবেই WordPiece training চলে।

তৈরি হওয়া **vocabulary**

শেষে print করলে দেখা যায়:

```
['[PAD]', '[UNK]', '[CLS]', '[SEP]', '[MASK]', ..., 'ab', '##fu', 'Fa', 'Fac', ...]
```

এখানে interesting ব্যাপার হলো:

WordPiece অনেক সময় BPE-এর তুলনায় word-এর useful অংশগুলো একটু দ্রুত শিখে নেয়।

যেমন:

- Fac
- Hug
- Hugg

এগুলো meaningful subword হয়ে গেছে।

একটা গুরুত্বপূর্ণ **note**

এখানে বলা হয়েছে:

`train_new_from_iterator()` use করলে exact একই vocab নাও আসতে পারে।

কারণ Hugging Face-এর library training-এর সময় pure WordPiece internally follow করে না; অনেক ক্ষেত্রে BPE-style approach use করে।

মানে, এই tutorial-এর code হচ্ছে WordPiece logic বোঝানোর জন্য approximate implementation।

এবার নতুন **word tokenize** করা হবে কীভাবে?

এই function সেটা করছে:

```
def encode_word(word):  
    tokens = []  
    while len(word) > 0:
```



```

i = len(word)
while i > 0 and word[:i] not in vocab:
    i -= 1
if i == 0:
    return ["[UNK]"]
tokens.append(word[:i])
word = word[i:]
if len(word) > 0:
    word = f"##{word}"
return tokens

```

এটা খুব important।

এই **function**-এর **logic**:

একটা word থেকে শুরু করে:

1. শুরু থেকে সবচেয়ে বড় subword খোঁজে, যেটা vocab-এ আছে
2. সেটা token হিসেবে নেয়
3. বাকি অংশে ## যোগ করে আবার একই কাজ করে
4. যদি কোনো subword-ই না পাওয়া যায়, তাহলে পুরো word → [UNK]

এটাই WordPiece-এর আসল tokenization logic।

Example 1: Hugging

```
print(encode_word("Hugging"))
```

Output:

```
['Hugg', '##i', '##n', '##g']
```

মানে:

- প্রথমে Hugg পাওয়া গেছে
- তারপর ##i
- তারপর ##n
- তারপর ##g

এভাবে tokenize হয়েছে।

Example 2: HOgging

```
print(encode_word("HOgging"))
```

Output:

```
[['UNK']]
```

কেন?

কারণ word-এর শুরু থেকে এমন valid subword sequence পাওয়া যায়নি যা পুরো word cover করতে পারে।

তাই পুরো word unknown হয়ে গেছে।

পুরো text tokenize করার function

```
def tokenize(text):  
    pre_tokenize_result = tokenizer._tokenizer.pre_tokenizer.pre_tokenize_str(text)  
    pre_tokenized_text = [word for word, offset in pre_tokenize_result]  
    encoded_words = [encode_word(word) for word in pre_tokenized_text]  
    return sum(encoded_words, [])
```

এখানে যা হচ্ছে:

1. আগে sentence-কে word/punctuation-এ ভাঙছে
2. প্রতিটি word-এর ওপর `encode_word()` চালাচ্ছে
3. সব token একসাথে join করে final list দিচ্ছে

Example: পুরো sentence tokenize করা

```
tokenize("This is the Hugging Face course!")
```

Output:

```
['Th', '##i', '##s', 'is', 'th', '##e', 'Hugg', '##i', '##n', '##g', 'Fac', '##e', 'c', '##o', '##u', '##r', '##s', '##e', ['UNK']]
```

এখানে:

- This → Th, ##i, ##s
- is → is

- the → th, ##e
- Hugging → Hugg, ##i, ##n, ##g
- Face → Fac, ##e
- course → ছোট ছোট subword-এ split হয়েছে
- ! → vocabulary-তে না থাকলে [UNK]